SVM Inference Pipeline with Horner RBF Kernel Approximation

Detailed Mathematical Explanation

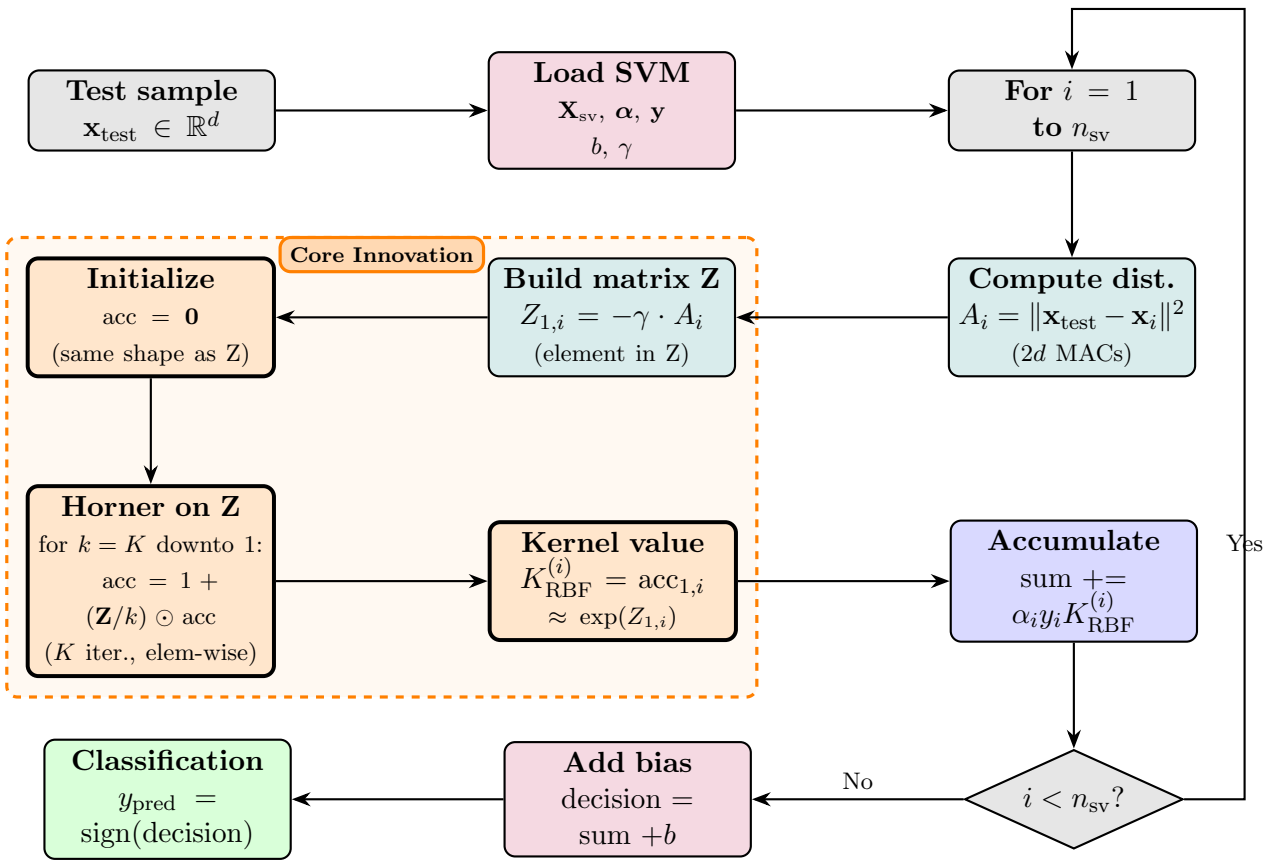
April 5, 2026

Abstract

This document presents the complete SVM inference pipeline with a focus on the Horner method approximation of the RBF kernel using matrix notation. We provide a detailed flowchart showing all computational steps from test input to classification output, followed by comprehensive mathematical explanations using element-wise Horner's method on the matrix **Z**.

1 SVM Inference Flowchart

The flowchart below illustrates the complete inference pipeline for a Support Vector Machine using the RBF kernel approximated via Horner's method applied element-wise to matrix **Z**.



Key Insight: The orange-highlighted section shows where Horner's method is applied element-wise to matrix $\mathbf{Z}$ to approximate the matrix exponential. For SVM inference, we build $\mathbf{Z}$ incrementally (one column per support vector) and extract the kernel values.

2 Detailed Inference Explanation

2.1 Input and Model Loading

Input: Test sample $\mathbf{x}_{\text{test}} \in \mathbb{R}^d$ is a d-dimensional feature vector.

SVM Model Parameters:

- **Support vectors:** $\mathbf{X}_{\text{sv}} \in \mathbb{R}^{n_{\text{sv}} \times d}$ - subset of training data defining the decision boundary
- **Dual weights:** $\boldsymbol{\alpha} \in \mathbb{R}^{n_{\text{sv}}}$ - Lagrange multipliers from dual optimization (only $\alpha_i > 0$ for support vectors)
- **Labels:** $\mathbf{y} \in \{-1, +1\}^{n_{\text{sv}}}$ - class labels of support vectors
- **Bias:** $b \in \mathbb{R}$ - decision hyperplane offset
- **Gamma:** $\gamma > 0$ - RBF kernel width parameter

SVM Decision Function (Dual Form):

$$f(\mathbf{x}_{\text{test}}) = \sum_{i=1}^{n_{\text{sv}}} \alpha_i y_i K_{\text{RBF}}^{(i)}(\mathbf{x}_{\text{test}}, \mathbf{x}_i) + b \tag{1}$$

2.2 Matrix Formulation

For SVM inference with a single test point, we conceptually build a row vector:

$$\mathbf{Z} \in \mathbb{R}^{1 \times n_{\text{sv}}} \tag{2}$$

where each element $Z_{1,i}$ corresponds to the i-th support vector. In practice, we can build this incrementally (one element at a time) or in batch (all at once).

2.3 Step 1: Compute Squared Distance Matrix A

For the single test point against all support vectors:

$$\mathbf{A} \in \mathbb{R}^{1 \times n_{\text{sv}}}, \quad A_{1,i} = \|\mathbf{x}_{\text{test}} - \mathbf{x}_i\|^2 = \sum_{j=1}^{d} (x_{\text{test},j} - x_{i,j})^2 \tag{3}$$

Cost: $n_{\text{sv}} \times 2d$ MACs total

Example: For $\mathbf{x}_{\text{test}} = [1.0, 2.0, 3.0]^T$, $\mathbf{x}_i = [1.5, 2.5, 3.5]^T$:

$$A_{1,i} = (1.0 - 1.5)^2 + (2.0 - 2.5)^2 + (3.0 - 3.5)^2 = 0.75 \tag{4}$$

2.4 Step 2: Compute Matrix Z

Apply element-wise multiplication with scalar γ:

$$\mathbf{Z} = -\gamma \mathbf{A} \in \mathbb{R}^{1 \times n_{\text{sv}}} \tag{5}$$

where $Z_{1,i} = -\gamma A_{1,i}$ for each support vector i.

2.5 Step 3: Horner's Method Applied to Matrix Z (Core Innovation)

2.5.1 RBF Kernel Matrix Definition

The exact RBF kernel matrix is:

$$\mathbf{K}_{\text{RBF}} = \exp^{\odot}(\mathbf{Z}) \tag{6}$$

where $\exp^{\odot}$ denotes element-wise (Hadamard) exponential.

2.5.2 Hadamard Series Expansion

The element-wise Taylor series for matrix $\mathbf{Z}$ is:

$$\exp^{\odot}(\mathbf{Z}) = \mathbf{1} + \mathbf{Z} + \frac{\mathbf{Z}^{\odot 2}}{2!} + \frac{\mathbf{Z}^{\odot 3}}{3!} + \cdots = \sum_{k=0}^{\infty} \frac{\mathbf{Z}^{\odot k}}{k!} \tag{7}$$

where:

- $\odot$ denotes the Hadamard (element-wise) product
- $\mathbf{Z}^{\odot k}$ means element-wise k-th power: $[\mathbf{Z}^{\odot k}]_{i,j} = Z_{i,j}^k$
- Division by $k!$ is element-wise: each element divided by the scalar $k!$

Truncate at K terms:

$$\mathbf{K}_{\text{RBF}} \approx \sum_{k=0}^{K} \frac{\mathbf{Z}^{\odot k}}{k!} \tag{8}$$

For $K = 12$: float32 precision (7 decimal digits)

2.5.3 Horner's Method Transformation

The key insight is to rewrite the series using Horner's nested form, applied element-wise:

$$\exp^{\odot}(\mathbf{Z}) \approx \mathbf{1} + \frac{\mathbf{Z}}{1} \odot \left(\mathbf{1} + \frac{\mathbf{Z}}{2} \odot \left(\mathbf{1} + \frac{\mathbf{Z}}{3} \odot \left(\cdots \left(\mathbf{1} + \frac{\mathbf{Z}}{K}\right)\right)\right)\right) \tag{9}$$

where division by scalars and the Hadamard product are applied element-wise.

2.5.4 Iterative Algorithm on Matrix Z

Algorithm 1 Horner's Method for $\exp^{\odot}(\mathbf{Z})$ with K terms

Require: $\mathbf{Z} = -\gamma\mathbf{A} \in \mathbb{R}^{m\times n}$, $K = 12$
Ensure: $\mathbf{K}_{\text{RBF}} \approx \exp^{\odot}(\mathbf{Z})$

```
acc ← 0_{m×n}                          ▷ Initialize accumulator matrix
for k = K down to 1 do
    acc ← 1 + (Z/k) ⊙ acc              ▷ Element-wise: K MACs per element
end for
return acc
```

Cost per element: Exactly K MAC operations

Total cost for matrix: $K \times m \times n$ MAC operations

For SVM inference with single test point: $m = 1$, $n = n_{\text{sv}}$, so total cost is $K \times n_{\text{sv}}$ MACs.

2.5.5 Worked Example (Single Element)

Consider one element $Z_{1,i} = -0.375$ (from $\gamma = 0.5$, $A_{1,i} = 0.75$), with $K = 4$:

$$k = 4: \quad \text{acc}_{1,i} = 1 + \frac{Z_{1,i}}{4} \cdot 0 = 1 + \frac{-0.375}{4} \cdot 0 = 1.0000 \tag{10}$$

$$k = 3: \quad \text{acc}_{1,i} = 1 + \frac{Z_{1,i}}{3} \cdot 1.0000 = 1 + \frac{-0.375}{3} \cdot 1.0000 = 0.8750 \tag{11}$$

$$k = 2: \quad \text{acc}_{1,i} = 1 + \frac{Z_{1,i}}{2} \cdot 0.8750 = 1 + \frac{-0.375}{2} \cdot 0.8750 = 0.8359 \tag{12}$$

$$k = 1: \quad \text{acc}_{1,i} = 1 + \frac{Z_{1,i}}{1} \cdot 0.8359 = 1 + (-0.375) \cdot 0.8359 = 0.6865 \tag{13}$$

Exact: $\exp(Z_{1,i}) = \exp(-0.375) = 0.6873$
Error ≈ 0.001 (0.1%) with $K = 4$. For $K = 12$: error $< 10^{-14}$.
This same operation happens **in parallel** for all elements of $\mathbf{Z}$.

2.5.6 Why Horner's Method is Superior

1. **Avoids factorial overflow:** Computing $k!$ directly fails for $k \geq 12$. Horner computes $\mathbf{Z}/k$ (element-wise division by scalar k).
2. **Reduces operations:** Direct series needs $\sim K^2$ ops per element. Horner: only K ops per element.
3. **Numerically stable:** Accumulates smallest to largest magnitude, minimizing rounding errors.
4. **Vectorizable:** Element-wise operations on $\mathbf{Z}$ can be parallelized across all matrix elements.
5. **Fixed latency:** Always K iterations, no branches—ideal for hardware pipelining.

2.6 Step 4: Extract Kernel Values

After Horner iteration on matrix $\mathbf{Z}$, extract kernel values:

$$K_{\text{RBF}}^{(i)} = \text{acc}_{1,i} \approx \exp(Z_{1,i}) = \exp(-\gamma A_{1,i}) \tag{14}$$

Error bound for each element:

$$|K_{\text{RBF}}^{(i)} - \exp(Z_{1,i})| < \frac{|Z_{1,i}|^{K+1}}{(K+1)!} \tag{15}$$

For normalized data with $\gamma \in [0.1, 10]$: error $< 10^{-7}$ (negligible).

2.7 Step 5: Accumulate Weighted Kernel

Accumulate: sum $+= \alpha_i \cdot y_i \cdot K_{\text{RBF}}^{(i)}$ (1 MAC operation)
After all n_{sv} support vectors:

$$\text{sum} = \sum_{i=1}^{n_{\text{sv}}} \alpha_i y_i K_{\text{RBF}}^{(i)} \tag{16}$$

2.8 Step 6: Loop Termination

Check: $i < n_{\text{sv}}$? If Yes: next i. If No: proceed to bias.

2.9 Step 7: Add Bias Term

$$\text{decision} = \text{sum} + b = \sum_{i=1}^{n_{\text{sv}}} \alpha_i y_i K_{\text{RBF}}^{(i)} + b \tag{17}$$

2.10 Step 8: Classification Output

$$y_{\text{pred}} = \text{sign}(\text{decision}) = \begin{cases} +1 & \text{if decision} > 0 \\ -1 & \text{if decision} \leq 0 \end{cases} \tag{18}$$

Confidence: |decision| (large $\Rightarrow$ high confidence, small $\Rightarrow$ near boundary)

3 Computational Cost Analysis

Example: $n_{\text{sv}} = 50$, $d = 100$, $K = 12$: Total $= 50 \times 213 + 1 = 10{,}651$ MACs

Operation	Count	Cost
Build distance matrix $\mathbf{A}$	1	$n_{\text{sv}} \times 2d$ MACs
Compute $\mathbf{Z} = -\gamma\mathbf{A}$	1	n_{sv} multiplications
Horner on $\mathbf{Z}$ (element-wise)	K iterations	$K \times n_{\text{sv}}$ MACs
Weighted accumulation	n_{sv}	n_{sv} MACs
Bias addition	1	1 addition
Total		$n_{\text{sv}}(2d + K + 1) + 1$ MACs

Table 1: Computational cost breakdown for SVM inference.

Platform	Kernel Computation	Latency
Software CPU (3 GHz)	exp() per element	∼80 cycles
	Per classification	∼4,000 cycles / 1.3 μs
Hardware ASIC (200 MHz)	Horner on $\mathbf{Z}$, $K = 12$	12 cycles
	Per classification	∼600 cycles / 3 μs
Speedup (throughput)		**∼22×**
Energy efficiency		**∼100-1000×**

Table 2: Hardware vs software performance.

3.1 Hardware vs Software Comparison

Hardware advantages: (1) Vectorized Horner on $\mathbf{Z}$: all elements in parallel, (2) Pipelining: one result/cycle after K-cycle startup, (3) Low power: MACs vs exp() circuits, (4) Deterministic timing

4 Numerical Considerations

4.1 Convergence Region

Horner approximation quality for elements of $\mathbf{Z}$ with $K = 12$:

$\lvert Z_{i,j}\rvert$ range	Error	Quality
< 2	$< 10^{-10}$	Excellent
$2 - 5$	$< 10^{-7}$	Good (float32)
$5 - 8$	$< 10^{-4}$	Acceptable
> 8	Diverges	Poor

Table 3: Convergence for different $|Z_{i,j}|$ values.

4.2 Ensuring Good Approximation

To keep all elements of $\mathbf{Z}$ in convergence region:

1. **Normalize features:** Scale to $[0, 1]$ or zero mean/unit variance
2. **Choose γ appropriately:** For normalized data, $\gamma \in [0.1, 1]$ works well
3. **Increase K if needed:** For larger γ or unnormalized data, use $K = 16$ or $K = 20$

4.3 Precision Analysis

For each element: $\frac{|\text{acc}_{i,j} - \exp(Z_{i,j})|}{\exp(Z_{i,j})} \leq \frac{|Z_{i,j}|^{K+1}}{(K+1)! \cdot \exp(Z_{i,j})}$

For $K = 12$, $\gamma = 0.5$, $A_{i,j} \in [0, 4]$: Max absolute error $< 10^{-7}$, Max relative error $< 10^{-6}$ (well within float32)

5 Summary

SVM inference using Horner's method on matrix $\mathbf{Z}$ provides:

1. **Mathematical rigor:** Element-wise Horner on $\mathbf{Z}$ with provable error bounds
2. **Computational efficiency:** K operations per element vs complex exp()
3. **Vectorization:** All elements of $\mathbf{Z}$ processed in parallel
4. **Hardware amenability:** Fixed latency, simple MACs, fully pipeline-able
5. **Practical accuracy:** Float32 precision with $K = 12$ terms
6. **Significant speedup:** 20-100× faster with hardware acceleration
7. **Energy efficiency:** 100-1000× better power consumption

Key innovation: Applying Horner's method element-wise to matrix $\mathbf{Z} = -\gamma\mathbf{A}$ replaces expensive element-wise exponentials with vectorizable MAC sequences, enabling efficient hardware for real-time SVM inference.